API Documentation

This document provides detailed documentation of the SDK's API endpoints and functions.

Table of Contents

1. ComplianceSDK
2. TransferHookService
3. Types
4. Error Handling

ComplianceSDK

The main interface for interacting with the KYC Compliance Protocol.

Constructor

```
new ComplianceSDK(config: ComplianceConfig)
```

Parameters

Parameter	Type	Description
config	ComplianceConfig	SDK configuration object

ComplianceConfig

```
interface ComplianceConfig {
  rpcUrl: string;           // RPC endpoint URL
  programId: PublicKey;     // KYC Compliance Program ID
  wallet?: Wallet;          // Wallet adapter (optional)
  commitment?: 'processed' | 'confirmed' | 'finalized';
  skipPreflight?: boolean;
}
```

Methods

registerUser

Register a new user with KYC verification.

```
async registerUser(params: RegisterUserParams): Promise<string>
```

Parameters

```
interface RegisterUserParams {
  wallet: PublicKey;        // User's wallet address
  kycLevel: KycLevel;       // KYC verification level
  countryCode: string;      // Country code (ISO 3166-1 alpha-2)
  expiryTimestamp: number;  // Expiry timestamp (Unix time)
}
```

Returns

`string` - Transaction signature

Example

```
const signature = await sdk.registerUser({
  wallet: userWallet,
  kycLevel: KycLevel.Standard,
  countryCode: 'US',
  expiryTimestamp: Math.floor(Date.now() / 1000) + 365 * 24 * 60 * 60,
});
```

---

checkComplianceStatus

Check compliance status for a wallet.

```
async checkComplianceStatus(wallet: PublicKey):
Promise<ComplianceStatus>
```

Returns

```
interface ComplianceStatus {
  wallet: string;
  status: KycStatus;
  kycLevel: KycLevel;
  countryCode: string;
  expiryTimestamp: number;
  registered: boolean;
}
```

Example

```
const status = await sdk.checkComplianceStatus(walletAddress);
console.log(status.status); // 'verified', 'pending', 'frozen', etc.
```

---

approveTransfer

Check if a transfer is allowed between two wallets.

```
async approveTransfer(
  source: PublicKey,
  destination: PublicKey,
  amount: BN
): Promise<TransferResult>
```

Returns

```
interface TransferResult {
  approved: boolean;
  reason?: string;
  sourceStatus?: ComplianceStatus;
  destinationStatus?: ComplianceStatus;
}
```

Example

```
const result = await sdk.approveTransfer(
  senderWallet,
  receiverWallet,
  new BN(1000000)
);

if (result.approved) {
  // Proceed with transfer
} else {
  console.log('Transfer blocked:', result.reason);
}
```

---

createCompliantMint

Create a new mint with KYC transfer hook enabled.

```
async createCompliantMint(
  authority: PublicKey,
  decimals: number = 9
): Promise<{ mint: PublicKey; signature: string }>
```

Example

```
const { mint, signature } = await sdk.createCompliantMint(
  adminWallet,
  6  // USDC decimals
);
```

---

freezeWallet

Freeze a wallet (admin only).

```
async freezeWallet(wallet: PublicKey): Promise<string>
```

---

unfreezeWallet

Unfreeze a wallet (admin only).

```
async unfreezeWallet(wallet: PublicKey): Promise<string>
```

---

activateCircuitBreaker

Activate the circuit breaker to halt all transfers (admin only).

```
async activateCircuitBreaker(): Promise<string>
```

---

deactivateCircuitBreaker

Deactivate the circuit breaker (admin only).

```
async deactivateCircuitBreaker(): Promise<string>
```

---

getRegistryInfo

Get compliance registry information.

```
async getRegistryInfo(): Promise<{
  authority: string;
  totalRegistered: number;
  paused: boolean;
  circuitBreakerActive: boolean;
}>
```

TransferHookService

Service layer for managing Solana Token Extension transfer hooks.

Constructor

```
new TransferHookService(
  connection: Connection,
  provider: AnchorProvider,
  programId: PublicKey
)
```

Methods

createMintWithTransferHook

```
async createMintWithTransferHook(
  authority: PublicKey,
  decimals: number = 9
): Promise<MintWithHookResult>
```

updateTransferHookProgramId

```
async updateTransferHookProgramId(mint: PublicKey): Promise<string>
```

getTransferHookInfo

```
async getTransferHookInfo(mint: PublicKey): Promise<{
  programId: PublicKey | null;
  authority: PublicKey | null;
}>
```

Types

KycStatus

```
enum KycStatus {
  Unverified = 'unverified',
  Pending = 'pending',
  Verified = 'verified',
  Frozen = 'frozen',
  Sanctioned = 'sanctioned',
}
```

KycLevel

```
enum KycLevel {
  None = 'none',
  Basic = 'basic',
  Standard = 'standard',
  Enhanced = 'enhanced',
  Institutional = 'institutional',
}
```

RiskLevel

```
enum RiskLevel {
  Low = 'low',
  Medium = 'medium',
  High = 'high',
  Critical = 'critical',
}
```

Error Handling

All SDK methods throw `ComplianceError` on failure.

Error Codes

Code	Name	Description
0x1700	WALLET_NOT_REGISTERED	Wallet not in registry
0x1701	KYC_EXPIRED	KYC status expired
0x1702	WALLET_FROZEN	Wallet is frozen
0x1703	SANCTIONED	On sanctions list
0x1704	AMOUNT_EXCEEDS_MAXIMUM	Amount too high
0x1705	VELOCITY_LIMIT_EXCEEDED	Too many transactions
0x1707	CIRCUIT_BREAKER_ACTIVE	Transfers halted
0x1710	RECEIVER_NOT_VERIFIED	Receiver not KYC'd
0x1711	SENDER_NOT_VERIFIED	Sender not KYC'd

Handling Errors

```
import { ComplianceError, ErrorCode } from '@gitdigital/solana-kyc-
sdk';

try {
  await sdk.registerUser(params);
} catch (error) {
  if (error instanceof ComplianceError) {
    switch (error.code) {
      case ErrorCode.WALLET_NOT_REGISTERED:
        // Handle unregistered wallet
        break;
      case ErrorCode.KYC_EXPIRED:
        // Handle expired KYC
        break;
      default:
        console.error('Error:', error.message);
    }
  }
}
```

Quick Reference

Full Example

```
import {
  ComplianceSDK,
  KycLevel,
  createComplianceSDK
} from '@gitdigital/solana-kyc-sdk';
import { Keypair, PublicKey } from '@solana/web3.js';

// Initialize SDK
const sdk = createComplianceSDK({
  rpcUrl: 'https://api.devnet.solana.com',
  programId: new
PublicKey('KYCCo7vM2uLkGzqH6XqKJp1TJK5JjK9jW8vY9xQz1P2'),
  commitment: 'confirmed',
});

// Register user
await sdk.registerUser({
  wallet: userWallet,
  kycLevel: KycLevel.Standard,
  countryCode: 'US',
  expiryTimestamp: Math.floor(Date.now() / 1000) + 365 * 24 * 60 * 60,
});

// Check status
const status = await sdk.checkComplianceStatus(userWallet);
console.log(status.status); // 'verified'

// Approve transfer
const result = await sdk.approveTransfer(sender, receiver, amount);
if (result.approved) {
  // Execute transfer
}
```